Modelling market-based decentralised management systems

P J Kearney and W Merlat

A centralised approach to management of distributed systems works well when applied to relatively small-scale systems. As the scale grows, so do the problems of making appropriate system-wide decisions and co-ordinating their execution. As a result, large centrally managed systems tend to be ponderous and rigid. Devolving decision-making responsibility to local units means that locally appropriate responses to changed circumstances can be made rapidly. However, it is quite possible that local decisions in different elements of the system will interfere adversely.

How does one co-ordinate local decisions in large systems without re-introducing central control and its associated problems? The decentralised approach adopted here is to craft the interactions between local decision-making elements (agents), in order that effective management of the overall system is an emergent result of repeated local decisions and interactions. The general solution to this problem is an ambitious long-term research goal. This paper presents some results that offer a way forward within a particular class of problem in which the interactions between agents can be modelled as sale and purchase of commodity items.

1. Introduction

The world the future enterprise will inhabit will be characterised by rapid change, diverse products and short windows of market opportunity. The successful enterprise will need to be highly adaptive, agile and responsive to customer needs. We are already seeing moves towards increased local autonomy, putting decision-making responsibility in the hands of managers who are close to the customer. Information technology has a big role to play in combining the benefits of small entrepreneurial autonomous business units, with the economies of scale of a large integrated corporate infrastructure.

Software is currently experiencing its own parallel crisis, however. One frequently reads horror stories of large software projects coming in late, over-budget, performing a function that is no longer needed, or just simply not working. A major factor contributing to this problem is the complexity of the specification required for large systems. This is exacerbated when they are expected to operate over long life times in an environment that could not be anticipated when their design began. This paper contends that beyond a certain system scale or complexity, it is more effective to specify the system in terms of relatively simple autonomous adaptive units (agents) and their local rules of interaction. Such a system should be easier to implement correctly, and be robust, flexible, expandable, etc. However, in order to ensure that the overall system behaves well, one needs to understand how the agent behaviours combine dynamically to generate the system behaviour.

The Dynamo[1] team within BT's Intelligent Business Systems Research (IBSR) group is working to develop such an understanding through computational experiments and mathematical analysis. The intention is to express this knowledge as engineering principles, methods and techniques for the specification of decentralised systems that are robust, adaptive and scaleable. The particular interest here is in systems in which the agents interact by buying and selling services, and hence form a market. Such an approach is advocated as a means of harnessing the self-seeking drives of autonomous agents to achieve collective benefit without the dead hand of authoritarian central control.

This paper is structured as follows. The following section looks more closely at decentralised management and the motivations for adopting such an approach. Agents are introduced as the basic decentralised management building block. Next it is argued that it is quite natural to view agent decisions and interactions in economic terms, and hence that decentralised systems are usefully modelled

[1] Dynamo stands for the Dynamics of Adaptive Market-based Organisations.

as economic systems or markets. This is not novel in itself, but serves to explain why a market-based model has been adopted within Dynamo. The WALRAS market-based control model due to Wellman is briefly reviewed as a backdrop against which to contrast this decentralised model. The remainder of the paper describes a particular abstract model used to explore the relationship between agent behaviour and interactions and the consequent system behaviour, and the results of simulation experiments with this model. While this paper adopts a purely economic model, related experiments have also been performed where evolutionary (i.e. genetic) and economic paradigms are combined. Some of these are reported in the paper by Smith et al [1]

2. Decentralised management

2.1 Motivations

BT's business, products, and network infrastructure abound with requirements for the management of distributed systems. In principle, and in an ideal world, a centralised approach to management offers considerable advantages — as all information is gathered together in one place at the same time, a solution can be formulated taking all known factors into account. In practice, however, there are problems. Often such a global management function is complex, which makes it difficult to specify, design, verify, etc. For similar reasons, it is difficult to change, leading to legacy problems and organisational inertia.

The traditional way to manage such complexity is to decompose the problem into weakly coupled sub-problems. These are considered individually, and then integrated to form the completed system. Here, this approach is labelled 'decompositional'. In principle, it works well if the sub-functions are independent to a good approximation, such that interdependencies can be treated as minor perturbations. This assumption frequently turns out not to be justified, and long development times and serious integration problems are the result. All of the above are exacerbated by increasing the size of the system being managed — centralised management approaches which work adequately on small systems are often found to work badly (or not at all) when the system is scaled up.

Furthermore, the 'correct' management function is often not known. Even if it is known when system analysis and design begins, the requirements are evolving, and the original specification may be out of date by the time the system enters service. The above discussion also ignores performance-related problems in gathering a temporally consistent set of data, fusing it into a coherent picture, making a decision, and propagating command instructions while they are still relevant to the state of the world.

A decentralised approach is essentially constructive rather than decompositional. Instead of starting by defining a complex system specification, then decomposing it, one defines the components and their interactions first. The overall system behaviour is then generated through the iterated interaction of the components. When this system behaviour is qualitatively different from a simple aggregation of the behaviours of the isolated components, it is described as 'emergent'. Extremely complex emergent behaviours can result from very simple component behaviours and interactions. It should be noted that such emergent effects arise in decompositional systems when the independence approximation is not justified, resulting in unanticipated (and usually undesired) system behaviour. A decentralised approach seeks to exploit rather than avoid emergence, however. To do this, an understanding must be developed of which component behaviours and interactions result in beneficial system behaviours. This requires a combination of experiment (using software simulations) and mathematical analysis.

Note that sometimes decentralised management is thrust upon us by practical reality, such as when different parts of the system in question are owned by independent or autonomous organisations. This is the case, for example with electronic commerce/supply chain networks, but also applies to systems that integrate component systems belonging to different organisational units within a company. Here, local autonomy is part of the requirement rather than a design choice. The challenge is to achieve a degree of synergy among sub-systems with different (possibly conflicting) goals without sacrificing autonomy.

2.2 Decentralisation and agents

A decentralised management/control system consists of a collection of relatively simple, autonomous, adaptive sub-functions, each associated with a unit of the controlled system. This concept of a local management unit is essentially the same as that of a software agent in a multi-agent system, and here the terms 'multi-agent system' and 'decentralised management system' will be treated as synonymous. Thus the question 'What is a decentralised system?' can be replaced by the one dreaded by all agent researchers — 'What is an agent?' This can be answered in three main ways, in terms of:

- implementation technology, e.g. an agent is a process implemented using an agent tool-kit, executing on an agent platform, or communicating using an agent communication language,
- software architecture/design, e.g. an agent has explicitly represented beliefs, desires and intentions (BDI architecture),

- modelling abstraction, e.g. something is an agent if it is useful to view it as having certain properties or qualities (autonomy, etc).

It is in this last sense that agent is used here. Of course, the three are related — if the problem specification is expressed in terms of agents, then it is reasonable to adopt a compatible agent-oriented design approach and implementation tool-kit. Notwithstanding, a decentralised system could also be designed using, say, an object-oriented method and implemented using Java or C++. The converse is also true — a system implemented using agent-based tools and design is not necessarily decentralised, indeed many (perhaps most) prototype and demonstration multi-agent systems are not.

Few people would violently disagree with the following list of key qualities defining an agent, though they might express them in different terms and add or delete one or two features. An agent:

- is situated in an environment, and it receives 'sensory' information from its environment, and its actions affect that environment,
- is both reactive and proactive — its actions can be triggered by external events that it 'senses', or by internal events (i.e. changes of its own internal state),
- is autonomous (and boundedly rational) — in deciding which actions to perform, its choice is based on the estimated benefit to itself and it therefore needs some notion of a goal or goals it aims to achieve or function it seeks to maximise; the choice is made using imperfect and incomplete local information (local information basically means an agent's own partial and uncertain model of its environment plus direct measurements of the current state of its immediate environment — one can think of the agent's world model as an initial model that is updated repeatedly based on sensory data and the agent's inferences),
- interacts (asynchronously) with other agents — an important part of an agent's environment is made up of other agents, with whom it can interact indirectly, by causing changes in the environment that can be observed by other agents (and vice versa), and it can also interact directly, often via messages (it should be noted that receipt of messages is a form of sensory input — similarly, sending a message is a form of action on an agent's environment),
- has finite resources and capabilities — the agent's resources for storing and reasoning about its world model are bounded; similarly, as the agent's capacity for performing actions in a given time-slot is limited, an agent must often choose between desirable actions even when they do not conflict on an intrinsic level.

Although not reliant on interaction, each agent normally interacts with a relatively small number of other agents (exchanging information, requesting and agreeing services, etc). The presence of interactions allows information to propagate throughout the system, while restricting the number of interaction partners preserves scalability properties.

Whereas the totally centralised approach gathers a complete information snapshot on which to base a global decision, a decentralised approach makes many (in general, asynchronous) local decisions. The effects of the local decisions impact on future decisions of neighbouring agents, which in turn influence their neighbours. In some cases the effects weaken with 'distance' (and time), but others are self-, or mutually, reinforcing so that truly global effects can be achieved from local decisions. However, as the influences propagate over 'distance' and time, precision is lost. An agent knows in detail about its own state and that of the unit it controls, it knows as much about its direct acquaintances as they tell it, and it feels the influence of more distant agents via their influence on the acquaintances, etc. Thus if an agent needs precise information about the states of a large proportion of other agents, then either the agents have been chosen badly, or else the problem is inherently centralised. However, it seems reasonable that in many cases agents can be chosen in such a way that this is not the case. Choosing agents' responsibilities and acquaintances based on the structure of the controlled system is a plausible basis for such an assignment, and is the approach taken in the current work.

2.3 Summary

Decentralisation therefore seems a promising approach to take on problems that are too complex to tackle in a centralised fashion, and which cannot be composed into sub-problems that can be solved independently. However, it is not a silver bullet. The approach depends on emergent system-level behaviours arising from the iterated actions and interactions. Such effects are, in general, difficult to predict and take time to emerge. This is a nettle that must be grasped, however.

It must be noted that benefits of a decentralised approach do not lie in enabling a system to meet a more demanding technical functional requirement, e.g. producing a highly optimised solution to a well-specified problem. Nor does it improve theoretical performance (in the sense of execution speed) or efficiency, e.g. by leading to an algorithm that yields results more quickly. In most practical problems for BT's business these are not the key requirements anyway. What is typically required is to achieve a good level of functionality and efficiency in a system that is:

- easy (and hence quick and cheap) to specify, implement and maintain,
- robust, i.e. it performs well over a wide range of conditions,
- adaptable, i.e. the system can adjust to changing requirements and operating conditions,
- scalable, i.e. the same architecture works well across a wide range of some scale parameter (e.g. number of nodes) — in particular, the architecture should continue to perform well as the scale parameter becomes very large,
- expandable/modifiable, i.e. new capacity or functionality can be added to (or removed from) the system easily,
- open (1 — heterogeneous), i.e. the system can be composed from modules from different sources and with different internal architectures,
- open (2 — fuzzy, dynamic boundaries to the system), e.g. in a supply chain management application, a given supplier's inventory management software could sometimes be part of the system, and sometimes not.

Having said that, decentralisation does not improve theoretical functionality and performance, it will often do so in practice. This can be because the alternative is too complex to realise in a timely manner (or at all), or because assumptions about the requirements or the operating conditions are erroneous.

3. Decentralised systems as economic systems

3.1 General

Given the foregoing description of decentralised systems as multi-agent systems, it is natural to introduce trading of services for money as an interaction between agents. Each agent controls only part of the underlying system, and so to achieve any larger objective, an agent is likely to need services under the control of other agents. For example, in a production line context, an agent might be managing one cell on the line. In order to step up production it needs the co-operation of the preceding cell (to supply the part-finished product), supplier of the components to be added, and the following cell (to take away the increased output).

Suppose agent *A* sends a request to agent *B* requesting that *B* performs some action of value to *A*. Since *B* decides what to do based on its own self-interest, it will not comply with that request if the anticipated net value to itself is negative. This will often be the case, since *B*'s capabilities are limited — performing the action requested by *A* will often mean delaying performing an action that *B* would really prefer to do. Thus *A* must persuade *B* by offering *B* something of value in return, so that the net value to *B* is positive. *A* barter system is possible, but introducing the concept of money is usually the simpler option. Co-operation is agreed between the two agents if the estimated value of the action to *A* minus the payment, and the estimated value of the action to *B* plus the payment, are both positive.

One may argue that agents should be community spirited, i.e. *B* should perform the requested action if it believes it is in the best interests of the community. The question is: How does *B* know? *B* has quite detailed and up-to-date knowledge of its local part of the system, and is best placed to judge local impact — this is exactly what *B*'s utility function reflects. Similarly, *A* is best placed to judge impact on its part of the system, and the value of the action is reflected in the price it is willing to pay. Thus in choosing between *A*'s request and its own selfish preference (or between competing requests from *A* and *C*) based on price, *B* is taking into account community interest. Now *A* could simply tell *B* its estimated value without any payment, but payment gives *A* an incentive to be accurate and truthful. It also encourages reciprocity. *B* could simply do a favour for *A* in the general expectation that *A* will return the favour later. However, payment helps ensure that the net value of the favours exchanged evens out[3].

In a decentralised system, no individual agent is in a position to know what is in the communal best interest — in fact one opts for the decentralised approach because it is impractical to make such an estimation. It is a tenet of economics that if an economic system is working efficiently, then price will evolve to reflect value — market forces (or the invisible hand of Adam Smith) adjust the going rate for a commodity until supply equals demand. One may question whether this happens in practice in real and artificial markets. Nevertheless, it is certainly true that prices and availability of services experienced locally by an agent do reflect to some extent the communal value of and demand for these services.

3.2 Relationship to other work

Of course, the idea that agents can interact by buying and selling services is not new. Trading-based co-ordination mechanisms, such as contract net, date from the early days of multi-agent systems and are quite commonplace. The purpose of the above discussion was to argue that it is quite natural to model decentralised systems as markets, and that little generality is lost in doing so. Having said that, most agent systems employing trading do not consider dynamic effects[2], and cannot really be considered as markets/economies.

More recently, a tradition of more truly market-based agent systems has built up. Clearwater [2] is a widely

[2] That is the decisions of the agents are only weakly coupled, e.g. the award of a contract to one agent does not have a major effect on the bids of agents for similar contracts at later times.

referenced collection of papers on market-based control. Michael Wellman of the University of Michigan [3] is the figure most identified with this field, having originated the term 'market-oriented programming'. Many of these systems, including Wellman's, adopt a neo-classical economic model. This takes as given that, in an efficient market, 'Adam Smith's invisible hand' will set the prices for goods such that supply and demand are equal. Systems such as Wellman's WALRAS embody the invisible hand in an 'auctioneer' following a tradition established by the economist, Walras. This gathers information from the various agents and sets an economic equilibrium price at which the next round of trading occurs. The existence of a unique equilibrium, and the ability of the auctioneer to find it, can be guaranteed provided the system has certain properties. One of these properties that is important to later discussion is that process steps do not display economies of scale — in other words, as the rate of processing increases, the unit cost must not decrease.

Within its domain of applicability, WALRAS appears to perform well. The following points, however, should be noted.

- The auctioneer operates outside normal time. It conducts iterated interactions with the agents in between trading 'days' to establish a set of equilibrium prices. These prices are then used for transactions and to determine agent behaviours during the following day.
- The auctioneer is clearly a centralised decision-making process. It gathers information from the agents to a central location and uses the global picture to set the prices that determine the behaviour implemented by the agents.
- Furthermore, the agents' behaviours depend on global information — the market prices. Update of these prices is a synchronising factor. In a continuously operating trading system (with the auctioneer operating inside normal time), the time taken to gather consistent information required for price setting, and then to propagate the new prices, will increase with the size of the system. There will be some system size above which the price-update time-scale will be greater than the time-scale on which agents have to make and act on decisions.
- If one is taking economic systems (rather than the Walrasian model) as the paradigm to inspire a computation model, it can be argued one needs to model its actual mechanisms. In a real economic system there is no price-setting auctioneer; the 'invisible hand' is an emergent phenomenon. The Walrasian auctioneer models the effects that this phenomenon is supposed to have, not the mechanism.
- The regime under which Walras' and Wellman's auctioneers are guaranteed to work is one which excludes the possibility of beneficial emergent effects such as self-organisation, as well as undesirable ones.
- Walrasian-style mechanisms aim to keep the system near economic equilibrium. This is not necessarily a good thing. It is not clear that many real economic systems operate near equilibrium. Interesting emergent effects in nonlinear dynamic systems occur far from equilibrium[3]. It is plausible that operation at equilibrium might increase the chances of getting stuck in false optima, and also reduce the ability of the system to adapt.

In contrast, the Dynamo system uses thoroughly decentralised models and aims to harness emergent market mechanisms without the idea of a centrally determined global market price. An extension to this work will be to go beyond the safe regime of the single equilibrium and explore self-organisation and related phenomena, though this will have to be done slowly and gradually. Paralleling this distinction between Wellman's and the Dynamo approach to multiagent systems is a similar schism in economics.

There is a sizeable minority of economists interested in modelling economies as dynamic systems operating away from equilibrium and displaying emergent properties. This grouping is usually associated with the Santa Fe Institute, in part because of two important workshops held there [4, 5]. Arthur [6] has referred to the general approach as complexity economics, an expressive term, but one that has yet to be accepted widely. As yet this 'Santa Fe school' appears to be a loose grouping of like-minded individuals — there is no corresponding 'Santa Fe model'.

4. Explorations in market-based management

The remainder of the paper describes experimental work to elucidate the issues discussed above. It builds on earlier work by Kearney [7], and has also been influenced by past and current research on the use of agents within process management within the IBSR Group at BT and its predecessors (see, for example, Jennings [8] and Odgers [9]).

The system to be managed is modelled as a collection of:

[3] Note that economic equilibrium is not quite the same concept of equilibrium in the physical and natural sciences.

- repositories for commodity goods — all items of a given commodity are regarded as identical, and a repository can only hold one type of item,
- flows — these transfer items between repositories without changing the type of item and conserving the item number,
- task engines — these are instantiations of what in economic modelling are known as 'technologies', where a technology is basically the ability to perform a task (in a particular way), and task engines change input item types into output item types, typically utilising resources and consuming money.

These elements allow a variety of important systems to be modelled. These include certain aspects of workflow applications and packet-switching networks. In the workflow case, tasks involve the processing of work items associated with orders. Think of a task as involving taking some paperwork (the work item) from an in-tray associated with the task, performing some work progressing the order, updating the work item and putting it in an out-tray. Processing the work item generally requires use of resources, both consumables and assets (including workers), and incurs a cost (think of money as a consumable resource).

4.1 The basic building blocks

The basic building block is a unit composed of an agent and the local region of the managed system for which it is responsible (a collection of linked task engines and repositories). All the agents are essentially similar. These building blocks connect via an interaction mechanism. This mechanism is intended, in conjunction with the behaviour rules of the agents, to cause the system to evolve to a state in which it is operating efficiently. Should circumstances change, it should evolve to the state appropriate to the new conditions. Ideally, one would like the mechanism to cope with as wide a range of configurations as possible. The research methodology used to develop and evaluate a mechanism starts by considering a single unit interacting with an environment that does not change its behaviour. This can be regarded as a kind of mean field approximation. Gradually more neighbours to the first agent are introduced, and the resultant behaviour analysed. The candidate mechanism described has developed as a result of several iterations through this process. Work is on-going, and so far only simulations containing small numbers of agents have been evaluated.

Two interacting basic units are shown in Fig 1. In the work described here, a managed unit contains a single task engine that transforms items of one type to items of another on a one-to-one basis. The task engine is associated with a stock pile for items of its input type, and optionally for its output type as well, and also a stock of money. The agent earns money by selling its output items, which it obtains by buying input items and transforming them at some cost. The agent makes a profit if its revenue from sales exceeds its production and purchase costs. The goal of the agent is to maximise its profit over the long term (i.e. it is interested in sustainable profit not short-term cash flow) in order to maximise the 'dividend' paid to its parent organisation. Summing the dividend payments of a group of agents gives a measure of collective success.

The agent influences its profit by altering the prices and amounts of its bids to buy and sell input and output items, and the level of production. The actual levels of all of these are not under the sole control of the agent, however. Sale transactions clearly require the existence and consent of one or more trading partners, for example. Achieving agreement on the price and amount of a sale can be thought of as negotiation, though in this work impartial 'auction' mechanisms are used to achieve the same end. These 'auctions' are the main interaction mechanism within the agent community.

In a workflow context this corresponds to a small functional unit of variable team size with an in-tray and out-tray of work items. Above the team, there are organisational levels. The intermediate level corresponds to management of a production unit (sharing a pool of 'workers'). The top level corresponds to a corporate level of management. In the approach presented here, the upper two levels are emergent consequences of decisions and interactions at the 'atomic'

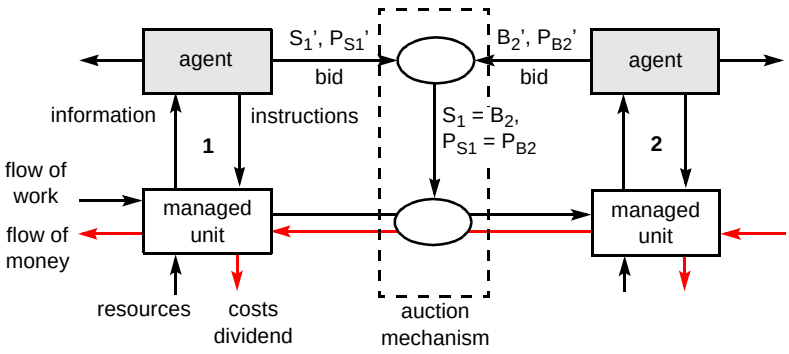

Fig 1 Two basic units interacting via an auction mechanism.

level. External to the organisation, there are also agents representing customers and suppliers.

The remainder of this section describes the various components and interactions in more detail. Firstly the functional units of the managed system are described. The management layer is then described. This is divided into an architectural or social element based on market mechanisms plus buy and sell bid formats, and a description of the agents' reactive rules of behaviour, which are driven by a value function. Some experimental results are shown for simple combinations of building blocks.

4.2 The managed system

It was mentioned above that the essential building block of the managed layer of the system is a task engine plus stockpiles. Stockpiles are simply repositories for items of a given type. The task engine needs further description, however. A task engine is an instance of a 'technology', embodying the ability to perform process steps that transform items of one type in one stockpile into the same number of items of another type in another stockpile. The rate at which this is done depends on the allocation of 'workers' to the step according to the formula:

$$P(w) = k_p w(1 + \alpha w) \quad \text{..... (1)}$$

where w is the number of workers, and k_p and α are constants. For simplicity all process steps use the same production function. The process step also consumes money from a stockpile at the rate:

$$C(w) = k_w w \quad \text{..... (2)}$$

i.e. workers receive a fixed wage, k_w, when being used, but not otherwise. Related quantities are productivity:

$$p(w) \equiv P(w)/w = k_p(1 + \alpha w) \quad \text{..... (3)}$$

and unit cost:

$$c(w) \equiv C(w)/P(w) = (k_w/k_p)/(1 + \alpha w) \quad \text{..... (4)}$$

The terminology and some aspects of the functions have been chosen to evoke workflow-style applications, but scenarios from other domains (e.g. manufacturing) can also be constructed from these components.

The information available to the controlling agent is restricted to:

- stock levels,
- rates of flow of items and money,
- actual worker allocations.

It should be noted that the management layer does not know the form of the equation (1), only its current value.

4.3 Market mechanism

The bids

The bids used in this work are piece-wise linear amount versus price curves[4]. Semantically, a sell (buy) bid curve represents a set of alternative point bids, each of which indicates a willingness to sell (buy) the stated amount at the stated price or higher (lower). Buy bids have the amount monotonically increasing (or constant) with increasing price, and sell bids decreasing. This form can be justified using arguments from microeconomic theory, and also gives the system useful dynamic properties. It is easy to see that increasing the amount offered for sale by shifting the sell bid upwards results in more being sold, but a lower unit price obtained. Similarly, shifting the selling bid upwards in price results in less being sold, but a higher price obtained.

In the current study, each curve has three segments, the slopes of which are the same for all buy and all sell bids. The end segments extend to infinity. The low price segment of sell bids, and high price segment of buy bids lie along the price axis. All other segments have non-zero slope. Because of these restrictions, an individual bid can be characterised by a single point representing the joint of the two non-zero segments of the bid curve (see Fig 2). It is also possible to regard this characterising point as the bid (with some implied tolerances), and the curves as part of a mechanism for achieving the best compromise.

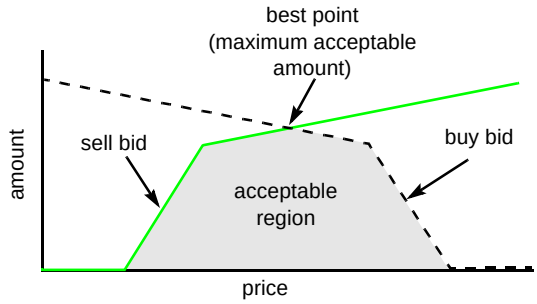

Fig 2 A pair of bids.

The auction mechanism

The term 'auction mechanism' is used here to denote an impartial mechanism that takes in one or more buy bids and one or more sell bids and yields a set of binding transactions, i.e. amounts and prices actually bought and sold by the bidders. This mechanism can be incorporated into a trading agent, in which case this agent is likely to be the sole buyer or seller, or embodied in an independent auctioneer. The basic mechanism involves calculating the

[4] Note that this is a specific case of a more general form of bid recently devised.

point at which a buy and sell bid curve intersect (again see Fig 2). This corresponds to the deal, acceptable to both parties, which maximises sales volume. It is assumed that the agents only bid to buy or sell in order that a profit is made on each item. This means that both seller and buyer are interested in maximising volume, while the seller wants to maximise price, and the buyer to minimise it. The intersection mechanism favours neither party over the other. When there is only one pair of bids for a given type of item, the transaction simply takes place at the price and amount given by this intersection point.

Two alternative methods are used for dealing with more than one possible pair of bids. The first of these involves calculating a price at which all transactions dealt with by this auctioneer during this time period take place. The amount bought or sold by each bidder is then simply the amount of its bid curve at this price. The price is calculated by combining all buy curves and all sell curves by summing amounts at corresponding prices, and then intersecting the summed curves as before. This will result in the total amount bought and sold being equal. In this method transactions effectively take place via a middle-man. This method resembles a single round of the iterative procedure used in Wellman's WALRAS.

The other method involves selecting bid pairs on the basis of sales volume as indicated by their intersection point. Here, each transaction takes place at a different price and is a direct deal between buyer and seller.

Worker limit

Two principal ways of managing the constraint of a finite number of workers in a production unit have been used:

- using a simple mechanism (e.g. normalisation) to adjudicate when the total number of workers requested by the agents within a production unit exceeds the number available,
- granting the number of workers requested, but increasing the cost of using a worker as the total used approaches the limit — this is analogous to the use of overtime payments.

The latter mechanism is currently preferred as it is more in line with the spirit of decentralisation. However, as it does allow the total number to be breached, the constraint is 'softened'. Given an appropriate employment cost function, any breach would normally be temporary, as an inefficient use of resources will penalise the competitiveness of the agent.

4.4 Agent behaviour rules

An agent's behaviour rules must adjust its bids and production level so that:

- input, production and output come into balance,
- the steady state thus achieved is the one that maximises cash flow.

These two sub-problems can be handled separately. The strategy adopted here is to use the production level and sell bid to maintain a steady state. The buy bid is used to adjust the input amount. If a steady state is maintained, this will have the effect of controlling the overall level of throughput.

Let us look at maximising steady-state cash flow. The agent behaviour rules are based on the assumption that its trading environment can be approximated by a single supplier and a single customer, each of which vary their bids only slowly. The bids are monotonic functions as described above. The level of cash flow for the agent under conditions of equal input, production and output has a simple maximum when plotted as a function of throughput. For example, if the bids are linear and the unit production cost constant, then the profit margin versus throughput curve is also linear (with negative slope). The cash flow (throughput X profit margin) is then a quadratic with a maximum where throughput is half its break-even value (see Fig 3). The obvious strategy for the agent in order to maximise cash flow, is to adjust its bids to increase throughput in proportion to the gradient of the cash-flow curve — this will seek out the maximum. Unfortunately, the agent cannot measure this gradient directly.

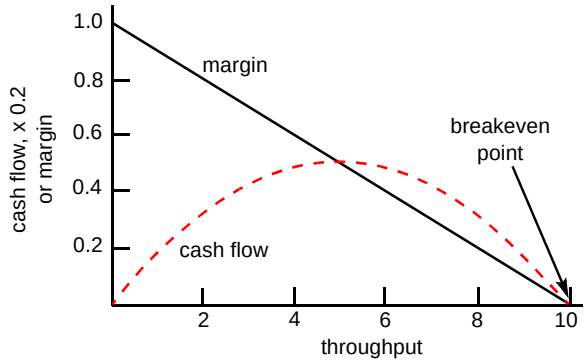

Fig 3 If profit margin is a linear function of throughput, then maximum profit occurs when throughput is half its breakeven value.

Instead the agent is given a target level of profit on each item sold (which it pays as dividend). The control problem is then to find the highest level of throughput at which this profit margin may be maintained. This simply involves trying to increase throughput in proportion to cash flow net of dividend payments. Note that this will only result in maximal profit if the target level of unit profit happens to be optimal.

Of course, the throughput cannot be chosen directly by the agent. By design, shifting the bid curves up or down in terms of amount results in a change in the amount sold of the same sign (given constant customer/supplier bids). Thus to control output (input), the agent shifts its sell (buy) bid.

The effects are felt through changes to input and output rates of goods and cash. The control strategy adopted in this work has been to alter the buy bid in response to cash flow changes, aiming to minimise the cash flow after subtraction of dividend (maximising throughput for a given minimum unit profit). The sell bid/production level are used in a similar fashion to minimise changes to the stock levels, and thus establish and maintain an equality among input, output and processing rates. Thus the buy bid is used to control throughput level, and the other degrees of freedom used to achieve the sustainability condition. In the current work, the price difference between reference points on the buy and sell bid curves is kept constant, and the curves simply shifted up and down in the price dimension in line with the mean of the actual prices achieved. This is only one among many possible strategies, however.

5. Results from simulation experiments

This section presents results obtained using simple combinations of the components and mechanisms described above.

5.1 A single agent

Simulation experiments confirm that the rules described above do indeed cause an agent in a static trading environment to reach a steady state with the profit margin at the target level. The rules also enable the agent to adapt to a changing environment. Suppose the agent has reached its static optimum throughput, and then the customer shifts its buy bid curve up (down) in amount. This has the effect of increasing (decreasing) both the amount sold and the unit price received by the agent, and increasing (decreasing) the optimum throughput level. The resultant change in cash flow causes the agent to increase (decrease) the amount of its buy bid, which in turn alters throughput in such a way as to track the movement of the optimum value. Similar experiments can be constructed for changes to the price of the customer's bid and for changes to the supplier's bid. So, the combination of bid structure, bid resolution mechanism and behaviour rules result in the agent optimising its own state, and subsequently tracking this optimal point as external conditions change.

5.2 Competition between two agents

Suppose a second agent is introduced, similar to the first and in competition with it, and that the 'market price' auction mechanism is used. The auction mechanism couples the environments of the two agents, even though they do not trade directly with each other. With this mechanism, bids are summed, and both agents pay (receive) the same unit price for their input (output) items. The agent with the higher bid (in amount terms) will acquire (sell) more items. How the system evolves subsequently, depends on whether the production functions are of the economies of scale or diseconomies of scale type. In the economies of scale case, the agent with the higher throughput has higher production costs. For a given input and output price, its unit profit is less, and in consequence it will increase its throughput more slowly (or decrease it more rapidly) than its competitor. This causes the states of the two agents to converge. In the diseconomies of scale case, the competitive pressure causes the states to diverge. Eventually, one agent has zero throughput, and the other has the whole market to itself. Thus the former case is an example of load balancing between alternative process pathways, and the latter is a continuous evolution form of selection of a single best process pathway. Note that in both cases, the system 'chooses' the most efficient overall solution.

5.3 A two-agent supply chain

If the last case is connecting two agents in parallel, then here they are connected in series, forming a supply chain between an external supplier and customer. The first agent consumes the input item type to produce an intermediate type, which is consumed by the second agent to produce the type of item sought by the external customer. The two agents are now linked not by a competitive relationship, but one of mutual interests. Neither agent can sustain a profit on its own. Each agent is now operating within a trading environment that changes in response to its actions. The rules of behaviour and market mechanism described above result in the two agents co-evolving to a state in which their throughputs are equal and at the highest level at which the target unit profit can be sustained.

5.4 Competition between supply chains

Two composite agents (or production) are now placed in competition. Each is a value chain as discussed in section 5.3, with agents within a chain sharing a pool of workers. The cost of employing the workers rises as the total number of workers employed within a production unit increases beyond a threshold value. As the agents can trade across production unit boundaries, the first agent in composite 1 (in Fig 4) competes with its equivalent in composite 2 to sell to the two second agents, thus producing the system shown.

It should be noted that there are four competing end-to-end processes involved — a job can go via either of the first two agents followed by either of the second two. A key factor in determining the most efficient balance of processes is the character of the production/cost functions of the task engines. If they exhibit diseconomies of scale (dos), i.e. if the productivity of the workers decreases as more workers are allocated to a task, then the most efficient configuration involves distributing work between alternative instances of end-to-end processes. On the other hand, if they exhibit economies of scale (eos), then the most efficient configuration involves concentrating work in a single process instance; more specifically, one of the diagonal process instances, as these allow full use of the workers of both production units.

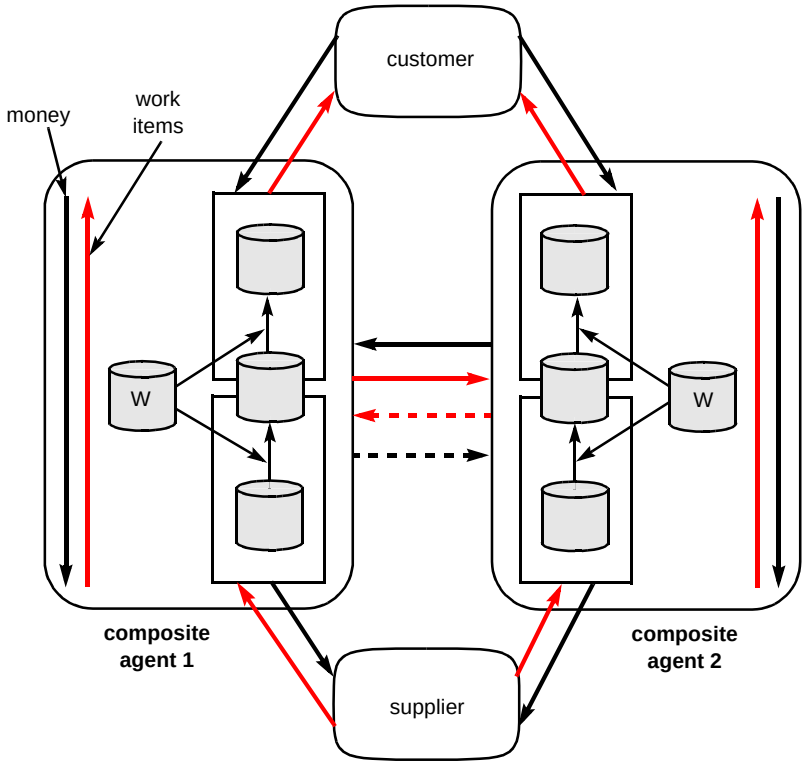

Fig 4 Two composite agents in competition.

Of these cases, 'economies of scale' is more challenging. Figure 5 shows the evolution of the states of the two composite agents using a state space defined by the number of workers allocated to each of the constituent agents. After an initial period of settling down, both composite agents reach a similar state with equal workers allocated to each step. They both increase their throughput (keeping the number of workers equal) until a limit is reached due to the increasing cost of workers. This phase is very similar to what happens in the dos case and also for a single composite agent in isolation.

The eos case involves a further stage — the competitive forces between process instances and the co-operative forces along process instances cause the composite agents to evolve in opposite directions in state space. The system eventually reaches the state that implements one of the two most efficient process instances.

In this final state the composite agents are in co-dependent states a long way from their normal operating states. Reaching these states involves co-ordinated behaviour on the part of the agents, yet such behaviour was not explicitly programmed into their behaviour rules. It is contended here that this co-adaptive self-organisation is an example of emergent behaviour, albeit a simple one. It is only the economies of scale case that engenders such 'interesting behaviour' (note that such production functions violate the conditions that guarantee convergence of the Walrasian price-setting mechanism).

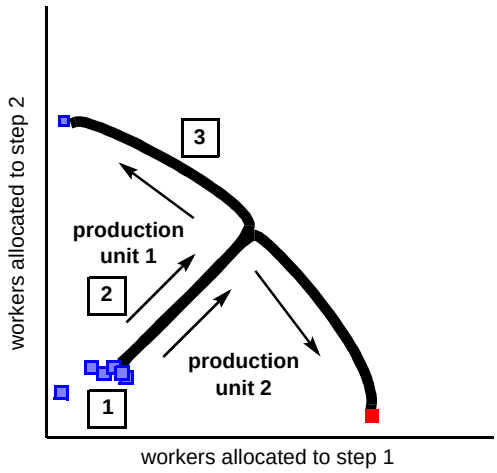

Fig 5 State space trace of competing composite agents (eos case).

6. Conclusions, related work and future directions

This paper has given an outline of the Dynamo activity. The main motivation for this work is the need to be able to specify decentralised management systems, and strong arguments have been presented as to why this is required. More specifically, a market-based approach is advocated as a means of harnessing the self-seeking drives of autonomous agents to achieve collective benefit without the dead hand of authoritarian central control. The major issue in decentralised systems is the double-edged sword of emergent behaviour, i.e. system behaviour that arises as a result of interactions between the behaviour of the system components. One wants a system that is flexible and

adaptive, which requires ceding a certain amount of control/predictability. This brings with it the potential for undesirable as well as desirable results (one man's adaptability is another man's instability). It is important to resist re-introducing centralised control to counter the undesirable behaviour, as one is then likely also to remove the potential for the desirable behaviour and also to introduce other problems. Instead qualities that inhibit the emergence of the undesirable behaviours must be built into the agent behaviours and interactions.

The second half of the paper described progress in the study of such issues using an abstract process model. This model is certainly not general, but is able to describe a class of systems corresponding to applications of interest to BT. It has proved a useful research vehicle in exhibiting emergent behaviours that are challenging yet tractable, and studying it has yielded considerable insight and some concrete results in terms of candidate market mechanisms. The potential and limitations of these mechanisms is still under investigation. Further research challenges will undoubtedly be thrown up as the number of agents is scaled up, especially using the 'economies of scale' production function.

Two other related studies have been performed within the Dynamo team in the last year. These introduce two elements into the mix:

- discrete changes to the set of active behaviour rules — this is a more strategic type of behaviour change than is involved in the study described in this paper,
- communication among agents regarding their current behaviour rule set and performance.

These factors are implemented using a decentralised evolutionary computing (EC) algorithm developed by Smith [10]. Smith first applied the EC algorithm to the market-oriented business process problem under a BT Short Term Fellowship with the IBSR Group [10]. The discrete changes involved here are binary decisions by an agent about which task engines it employs, and take place on a time-scale much longer than the reactive responses to market conditions. The other study takes an initial look at strategies regarding:

- speed of reaction to market conditions,
- the extent to which extrapolated market trends are taken into account in current decisions.

Only simplistic economic systems (composed of a few types of work items and a few task engines) have been studied so far.

Future work is planned to continue in two main directions. The first of these is to continue research along the lines presented here, generalising the techniques, looking at alternative process models, combining market-based and biologically inspired approaches, etc. This strand will largely be pursued in collaboration with universities and other partners. The second is to begin applying the essence of this approach to business problems of importance to BT. To this end a number of subjects have been selected for case studies. These are being pursued in the current year and serve both to test the Dynamo philosophy, and to help in disseminating these ideas.

References

1 Smith R E, Kearney P J and Merlat W: 'Evolutionary adaptation in autonomous agent systems: a paradigm for the emerging enterprise', BT Technol J, 17, No 4, pp 157—167 (October 1999).

2 Clearwater S (Ed): 'Market-based control: a paradigm for distributed resource allocation', World Scientific (1996).

3 Wellman M — home page at University of Michigan — http://ai.eecs.umich.edu/people/wellman/

4 Anderson P W, Arrow K J and Pines D (Eds): 'The economy as an evolving complex system', Santa Fe Institute /Addison Wesley (1988).

5 Arthur W B, Durlauf S N and Lane D (Eds): 'The economy as an evolving complex system II', Santa Fe Institute/Addison Wesley (1997).

6 Arthur B W: 'Complexity and the economy', Science, 284, (April 1999).

7 Kearney P J, Sehmi A and Smith R M: 'Emergent behaviour in a multi-agent economic simulation', in 11th European Conference on Artificial Intelligence (1994).

8 Jennings N, Faratin P, Johnson M J, Norman T, O'Brien P D and Wiegand M E: Agent-based process management', Journal of Cooperating Information Systems (January 1997).

9 Odgers B R, Shepherdson J W and Thompson S G: 'Distributed workflow co-ordination by proactive software agents', in Proceedings of Intelligent Workflow and Process Management, AAAI-99 Workshop, Orlando, Florida (August 1999), and other papers listed at http://www.labs.bt.com/projects/ibsr/papers/aew/papers.htm

10 Smith R E and Taylor N: 'A framework for evolutionary computation in agent-based systems', in Proceedings of the 1998 International Conference on Intelligent Systems (Looney C and Castaing J (Eds)) ISCA Press, pp 221—224 (1998).

Paul Kearney is a Senior Research Fellow working in the Intelligent Business Systems Research (IBSR) Group at BT Laboratories, Adastral Park. He has a BSc in Mathematical Physics (Liverpool University) and PhD in theoretical particle physics (Durham University). He spent nearly 10 years working for British Aerospace (Military Aircraft), firstly in aerodynamics, then progressing through graphics programming and then expert systems to R&D in on-board IKBS (tactical decision aids, etc) and software engineering. Joining Sharp Laboratories of Europe on 1990, he led a small research group applying agent concepts to personal electronics. He moved to BT in 1997. A particular research interest is in how complex collective behaviour patterns arise through the interaction of autonomous agents. His current work in the IBSR group pursues this interest in the context of providing support for business processes in a changing environment. He is BT's representative in PLANET, the European Network of Excellence in AI Planning and Scheduling, heading the technical coordination unit on Workflow Management, and on the Eurescom Project, MESSAGE (Methodology for Engineering Systems of software Agents).

Walter Merlat joined BT Intelligent Business Systems Research group in November 1997. Since then his work has focused on market-based control mechanisms for business processes and telecommunications networks.

He holds an MSc and a PhD in Artificial Intelligence from the Université Pierre et Marie Curie in Paris, France.

He is also a Chartered Telecom Engineer of the French Institut National des Telecommunications.